Deep Learning

Neural networks from the perceptron to modern architectures, with PyTorch and Keras code.

Contents

#	Topic	Level
1	Perceptron and neuron	Beginner
2	Activation functions	Beginner
3	Feed-forward networks (MLP) and the universal approximation idea	Beginner
4	Loss functions	Beginner
5	Forward propagation and backpropagation	Intermediate
6	Neural network from scratch in NumPy	Intermediate
7	PyTorch basics: tensors, autograd, nn.Module, DataLoader	Beginner
8	Training loop in PyTorch	Beginner
9	Keras / TensorFlow basics	Beginner
10	Optimisers: SGD, Momentum, RMSprop, Adam, AdamW; LR schedules	Intermediate
11	Weight initialisation, vanishing/exploding gradients	Intermediate
12	Regularisation: L2, Dropout, BatchNorm, LayerNorm, data augmentation, early stopping	Intermediate
13	Convolutional Neural Networks (CNN)	Intermediate
14	Famous CNN architectures: LeNet, AlexNet, VGG, ResNet, Inception, EfficientNet, ConvNeXt	Advanced
15	Transfer learning and fine-tuning	Intermediate
16	Recurrent Neural Networks (RNN)	Intermediate
17	LSTM and GRU	Intermediate
18	Sequence-to-sequence, encoder-decoder and attention	Advanced
19	Transformers: self-attention, multi-head, positional encoding	Advanced
20	Autoencoders, VAE	Advanced
21	Generative Adversarial Networks (GAN)	Advanced
22	Diffusion models	Advanced
23	Graph Neural Networks (GNN)	Advanced
24	Self-supervised and contrastive learning	Advanced
25	GPU training: mixed precision, multi-GPU, DDP, FSDP	Advanced
26	Model compression: pruning, quantisation, distillation	Advanced
27	Debugging and best practices for training	Intermediate
28	Deep learning projects ladder	Intermediate

1. Perceptron and neuron [Beginner]

Explanation (English)

A neuron computes y = f(w.x + b). The perceptron uses a step function and can learn linearly separable problems.

Tanglish (Tamil + English) Vilakkam

Neuron = weighted sum + bias + activation. Perceptron simple classifier.

Example / Program

```
import numpy as np
w=np.array([1,1]);b=-1.5
for x in [(0,0),(0,1),(1,0),(1,1)]: print(x,int(np.dot(w,x)+b>0))  # AND gate
```

Learn more

YouTube search: Perceptron and neuron tutorial

2. Activation functions [Beginner]

Explanation (English)

Sigmoid, tanh, ReLU, Leaky ReLU, GELU, SiLU/Swish, Softmax. ReLU/GELU avoid vanishing gradients; softmax converts logits to probabilities.

Tanglish (Tamil + English) Vilakkam

Activation non-linearity kudukkum. ReLU hidden layers-la, softmax output-la.

Example / Program

```
import numpy as np
x=np.linspace(-3,3,5)
print(1/(1+np.exp(-x)),np.tanh(x),np.maximum(0,x))
```

Learn more

YouTube search: Activation functions tutorial

3. Feed-forward networks (MLP) and the universal approximation idea [Beginner]

Explanation (English)

Stacked dense layers learn hierarchical features; with non-linear activations an MLP can approximate any continuous function.

Tanglish (Tamil + English) Vilakkam

MLP = dense layers stack. Non-linear activation irundha edhayum approximate pannum.

Example / Program

```
import torch,torch.nn as nn
m=nn.Sequential(nn.Linear(4,16),nn.ReLU(),nn.Linear(16,3));print(m(torch.randn(2,4)).shape)
```

Learn more

YouTube search: Feed-forward networks (MLP) and the universal approximation idea tutorial

4. Loss functions [Beginner]

Explanation (English)

MSE/MAE/Huber for regression; binary cross-entropy, categorical cross-entropy for classification; hinge, focal, triplet/contrastive, CTC for special tasks.

Tanglish (Tamil + English) Vilakkam

Regression-ku MSE, classification-ku cross-entropy.

Example / Program

```
import torch,torch.nn as nn
print(nn.CrossEntropyLoss()(torch.randn(3,5),torch.tensor([0,2,4])))
```

Learn more

YouTube search: Loss functions tutorial

5. Forward propagation and backpropagation [Intermediate]

Explanation (English)

Forward pass computes predictions; backprop applies the chain rule layer by layer to compute gradients of the loss wrt every weight. Autograd automates it.

Tanglish (Tamil + English) Vilakkam

Forward pass prediction, backward pass gradient. Chain rule dhaan base.

Example / Program

```
import numpy as np
x=np.array([1.,2.]);w=np.array([.1,.2]);y=1.
p=w@x;loss=(p-y)**2;grad=2*(p-y)*x;w-=0.1*grad;print(loss,w)
```

Learn more

YouTube search: Forward propagation and backpropagation tutorial

6. Neural network from scratch in NumPy [Intermediate]

Explanation (English)

Implementing layers, forward/backward and SGD manually gives true understanding. Train on XOR / MNIST.

Tanglish (Tamil + English) Vilakkam

NumPy-la scratch-la NN ezhudhina concept clear aagum.

Example / Program

```
import numpy as np
rng=np.random.default_rng(0)
X=np.array([[0,0],[0,1],[1,0],[1,1]]);y=np.array([[0],[1],[1],[0]])
W1=rng.normal(size=(2,4));W2=rng.normal(size=(4,1))
sig=lambda z:1/(1+np.exp(-z))
for _ in range(5000):
    h=sig(X@W1);o=sig(h@W2)
    d2=(o-y)*o*(1-o);d1=d2@W2.T*h*(1-h)
    W2-=h.T@d2;W1-=X.T@d1
print(o.round(2).ravel())
```

Learn more

YouTube search: Neural network from scratch in NumPy tutorial

7. PyTorch basics: tensors, autograd, nn.Module, DataLoader [Beginner]

Explanation (English)

Tensors on CPU/GPU, requires_grad for autograd, nn.Module for models, Dataset/DataLoader for batching.

Tanglish (Tamil + English) Vilakkam

Tensor, autograd, nn.Module, DataLoader - PyTorch basics.

Example / Program

```
import torch,torch.nn as nn
x=torch.arange(6.).reshape(2,3);print(x.device,x.sum(0));print(torch.cuda.is_available())
```

Learn more

YouTube search: PyTorch basics: tensors, autograd, nn.Module, DataLoader tutorial

8. Training loop in PyTorch [Beginner]

Explanation (English)

For each epoch/batch: zero grad, forward, compute loss, backward, optimiser step; validate with model.eval() and torch.no_grad().

Tanglish (Tamil + English) Vilakkam

Training loop: zero_grad, forward, loss, backward, step.

Example / Program

```
import torch,torch.nn as nn
X=torch.randn(100,3);y=X@torch.tensor([1.,2.,3.])+.1
m=nn.Linear(3,1);o=torch.optim.Adam(m.parameters(),.05)
for e in range(200):
    o.zero_grad();l=nn.functional.mse_loss(m(X).squeeze(),y);l.backward();o.step()
print(l.item(),m.weight)
```

Learn more
YouTube search: Training loop in PyTorch tutorial

9. Keras / TensorFlow basics [Beginner]

Explanation (English)
Sequential / Functional API, compile (optimizer, loss, metrics), fit, evaluate, predict, callbacks (EarlyStopping, ModelCheckpoint).

Tanglish (Tamil + English) Vilakkam
Keras easy API: compile, fit, evaluate.

Example / Program
```
import tensorflow as tf
(xt,yt),_=tf.keras.datasets.mnist.load_data()
m=tf.keras.Sequential([tf.keras.layers.Flatten(),tf.keras.layers.Dense(128,'relu'),tf.keras.layers.Dense(10)])
m.compile('adam',tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),['accuracy'])
m.fit(xt/255,yt,epochs=1)
```

Learn more
YouTube search: Keras / TensorFlow basics tutorial

10. Optimisers: SGD, Momentum, RMSprop, Adam, AdamW; LR schedules [Intermediate]

Explanation (English)
Momentum smooths updates; Adam adapts per-parameter rates; AdamW decouples weight decay. Use warm-up, cosine decay, ReduceLROnPlateau.

Tanglish (Tamil + English) Vilakkam
Adam default choice. LR scheduler use pannunga.

Example / Program
```
import torch,torch.nn as nn
m=nn.Linear(2,1);o=torch.optim.AdamW(m.parameters(),1e-3,weight_decay=.01);s=torch.optim.lr_scheduler.CosineAnnealingLR(o
```

Learn more
YouTube search: Optimisers: SGD, Momentum, RMSprop, Adam, AdamW; LR schedules tutorial

11. Weight initialisation, vanishing/exploding gradients [Intermediate]

Explanation (English)
Xavier/Glorot and He initialisation keep activations stable; gradient clipping fixes explosions; residual connections and normalisation help depth.

Tanglish (Tamil + English) Vilakkam
He/Xavier init. Gradient clipping explosion-a thadukkum.

Example / Program
```
import torch,torch.nn as nn
l=nn.Linear(10,10);nn.init.kaiming_normal_(l.weight);nn.utils.clip_grad_norm_(l.parameters(),1.0)
```

Learn more
YouTube search: Weight initialisation, vanishing/exploding gradients tutorial

12. Regularisation: L2, Dropout, BatchNorm, LayerNorm, data augmentation, early stopping [Intermediate]

Explanation (English)
Dropout randomly zeroes units; BatchNorm/LayerNorm stabilise and speed training; augmentation expands data; early stopping halts when validation loss rises.

Tanglish (Tamil + English) Vilakkam
Dropout, BatchNorm, early stopping - overfitting kuraikka.

Example / Program
```
import torch,torch.nn as nn
```

```
m=nn.Sequential(nn.Linear(10,64),nn.BatchNorm1d(64),nn.ReLU(),nn.Dropout(.3),nn.Linear(64,2))
```

Learn more
YouTube search: Regularisation: L2, Dropout, BatchNorm, LayerNorm, data augmentation, early stopping tutorial

13. Convolutional Neural Networks (CNN) [Intermediate]

Explanation (English)
Convolution layers with shared filters detect local patterns (edges -> textures -> objects); pooling reduces size. Few parameters thanks to weight sharing.

Tanglish (Tamil + English) Vilakkam
CNN image-ku. Filters edges, shapes kandupidikkum. Pooling size kuraikkum.

Example / Program
```
import torch,torch.nn as nn
m=nn.Sequential(nn.Conv2d(1,8,3),nn.ReLU(),nn.MaxPool2d(2),nn.Flatten(),nn.Linear(8*13*13,10));print(m(torch.randn(2,1,28
```

Learn more
YouTube search: Convolutional Neural Networks (CNN) tutorial

14. Famous CNN architectures: LeNet, AlexNet, VGG, ResNet, Inception, EfficientNet, ConvNeXt [Advanced]

Explanation (English)
Evolution: deeper (VGG), parallel filters (Inception), skip connections (ResNet), compound scaling (EfficientNet), modernised CNN (ConvNeXt).

Tanglish (Tamil + English) Vilakkam
ResNet skip connections vanishing gradient problem solve pannuchu.

Example / Program
```
import torchvision
m=torchvision.models.resnet18(weights=None);print(sum(p.numel() for p in m.parameters()))
```

Learn more
YouTube search: Famous CNN architectures: LeNet, AlexNet, VGG, ResNet, Inception, EfficientNet, ConvNeXt tutorial

15. Transfer learning and fine-tuning [Intermediate]

Explanation (English)
Reuse a pretrained model: freeze the backbone, train a new head, then optionally unfreeze with a small LR. Needs far less data and compute.

Tanglish (Tamil + English) Vilakkam
Pretrained model reuse. Backbone freeze, head train.

Example / Program
```
import torchvision,torch.nn as nn
m=torchvision.models.resnet18(weights='DEFAULT')
for p in m.parameters(): p.requires_grad=False
m.fc=nn.Linear(512,5)
```

Learn more
YouTube search: Transfer learning and fine-tuning tutorial

16. Recurrent Neural Networks (RNN) [Intermediate]

Explanation (English)
Process sequences step by step with a hidden state; suffers from vanishing gradients on long sequences. Used for time series, text, speech.

Tanglish (Tamil + English) Vilakkam
RNN sequence data-ku - hidden state munnadi information vaichirukkum.

Example / Program
```
import torch,torch.nn as nn
r=nn.RNN(5,8,batch_first=True);o,h=r(torch.randn(2,7,5));print(o.shape,h.shape)
```

Learn more
YouTube search: Recurrent Neural Networks (RNN) tutorial

17. LSTM and GRU [Intermediate]

Explanation (English)
Gated cells (input, forget, output gates) preserve long-term information. GRU is a lighter variant.

Tanglish (Tamil + English) Vilakkam
LSTM/GRU long-term memory vachukkum gates use pannum.

Example / Program
```
import torch,torch.nn as nn
l=nn.LSTM(5,8,num_layers=2,batch_first=True);o,(h,c)=l(torch.randn(2,7,5));print(o.shape)
```

Learn more
YouTube search: LSTM and GRU tutorial

18. Sequence-to-sequence, encoder-decoder and attention [Advanced]

Explanation (English)
Encoder compresses input, decoder generates output; attention lets the decoder look at all encoder states instead of one bottleneck vector.

Tanglish (Tamil + English) Vilakkam
Seq2Seq: translation maadhiri tasks. Attention = important parts-a focus pannum.

Example / Program
```
import numpy as np
q=np.random.rand(1,4);K=np.random.rand(5,4)
s=q@K.T;a=np.exp(s)/np.exp(s).sum();print(a)
```

Learn more
YouTube search: Sequence-to-sequence, encoder-decoder and attention tutorial

19. Transformers: self-attention, multi-head, positional encoding [Advanced]

Explanation (English)
Self-attention lets every token attend to every other token in parallel: Attention(Q,K,V)=softmax(QK^T/sqrt(d))V. Positional encodings add order. Basis of BERT, GPT, ViT.

Tanglish (Tamil + English) Vilakkam
Transformer = self-attention + feed forward + residual + layernorm. Parallel training - RNN-a vida fast.

Example / Program
```
import torch,torch.nn as nn
l=nn.TransformerEncoderLayer(d_model=32,nhead=4,batch_first=True);print(nn.TransformerEncoder(l,2)(torch.randn(2,10,32)).
```

Learn more
YouTube search: Transformers: self-attention, multi-head, positional encoding tutorial
Attention Is All You Need: https://arxiv.org/abs/1706.03762

20. Autoencoders, VAE [Advanced]

Explanation (English)
Autoencoders compress and reconstruct data (denoising, anomaly detection); VAEs learn a probabilistic latent space for generation.

Tanglish (Tamil + English) Vilakkam
Autoencoder compress & reconstruct. VAE new data generate pannum.

Example / Program
```
import torch,torch.nn as nn
enc=nn.Linear(784,32);dec=nn.Linear(32,784);x=torch.randn(4,784);print(((dec(enc(x))-x)**2).mean())
```

Learn more
YouTube search: Autoencoders, VAE tutorial

21. Generative Adversarial Networks (GAN) [Advanced]

Explanation (English)
A generator and discriminator play a min-max game; produce realistic images. Variants: DCGAN, StyleGAN, CycleGAN, Pix2Pix. Training is unstable.

Tanglish (Tamil + English) Vilakkam
GAN: generator fake create, discriminator fake-a kandupidikkum. Rendu compete pannum.

Learn more
YouTube search: gan pytorch tutorial dcgan

22. Diffusion models [Advanced]

Explanation (English)
Learn to reverse a gradual noising process; power Stable Diffusion, DALL-E, Sora-style video. Use U-Net / DiT denoisers with text conditioning.

Tanglish (Tamil + English) Vilakkam
Noise add panni, adha reverse-la remove panna kathukkum. Stable Diffusion idhu dhaan.

Example / Program
```
from diffusers import StableDiffusionPipeline
# p=StableDiffusionPipeline.from_pretrained('runwayml/stable-diffusion-v1-5');p('a cat').images[0]
```

Learn more
YouTube search: diffusion models explained

23. Graph Neural Networks (GNN) [Advanced]

Explanation (English)
Message passing over nodes/edges: GCN, GraphSAGE, GAT. For molecules, social networks, recommendation.

Tanglish (Tamil + English) Vilakkam
Graph data-ku GNN - nodes message pass pannum.

Example / Program
```
from torch_geometric.nn import GCNConv  # layer=GCNConv(16,32)
```

Learn more
YouTube search: Graph Neural Networks (GNN) tutorial

24. Self-supervised and contrastive learning [Advanced]

Explanation (English)
SimCLR, MoCo, BYOL, CLIP, masked autoencoders learn representations without labels.

Tanglish (Tamil + English) Vilakkam
Label illama representation learn - SimCLR, CLIP.

Learn more
YouTube search: self supervised learning contrastive simclr clip

25. GPU training: mixed precision, multi-GPU, DDP, FSDP [Advanced]

Explanation (English)
AMP (fp16/bf16) speeds training and saves memory; DistributedDataParallel scales across GPUs; FSDP/DeepSpeed shard huge models; gradient accumulation and checkpointing save memory.

Tanglish (Tamil + English) Vilakkam
Mixed precision, DDP, FSDP - big model train panna.

Example / Program
```
scaler=torch.amp.GradScaler()
with torch.autocast('cuda',dtype=torch.bfloat16):
    pass
```

Learn more

YouTube search: GPU training: mixed precision, multi-GPU, DDP, FSDP tutorial

26. Model compression: pruning, quantisation, distillation [Advanced]

Explanation (English)

Reduce size/latency: prune weights, quantise to int8/int4, distil a big teacher into a small student.

Tanglish (Tamil + English) Vilakkam

Pruning, quantization, distillation - model small & fast.

Example / Program

```
import torch
m=torch.nn.Sequential(torch.nn.Linear(8,8))
q=torch.ao.quantization.quantize_dynamic(m,{torch.nn.Linear},dtype=torch.qint8)
```

Learn more

YouTube search: Model compression: pruning, quantisation, distillation tutorial

27. Debugging and best practices for training [Intermediate]

Explanation (English)

Overfit one batch first; check shapes, label leakage, LR, normalisation; monitor train/val curves; log with TensorBoard/W&B; seed everything.

Tanglish (Tamil + English) Vilakkam

Mudhalla oru batch overfit pannunga. Shapes, LR, loss curves check.

Example / Program

```
import torch;torch.manual_seed(0)
# overfit single batch to sanity-check pipeline
```

Learn more

YouTube search: Debugging and best practices for training tutorial

28. Deep learning projects ladder [Intermediate]

Explanation (English)

MNIST digits -> CIFAR-10 CNN -> transfer learning on custom images -> sentiment LSTM -> text Transformer -> GAN faces -> diffusion fine-tune.

Tanglish (Tamil + English) Vilakkam

MNIST -> CIFAR -> custom images -> sentiment -> GAN. Padi padiya.

Learn more

YouTube search: deep learning projects for beginners